Java Data Structures

What is a Data Structure?

Data Structure:

A way of organizing and storing data in a computer so that it can be accessed and modified efficiently.

Two main groups in Java:

- **Linear:** Array, ArrayList, LinkedList, Stack, Queue
- **Non-linear:** Tree, Graph, HashMap, HashSet

Java Collections Framework (java.util)

Provides ready-made interfaces and classes for common data structures.

Array

- An array is a **contiguous block of memory** holding a fixed number of **homogeneous elements** (all of the same type).
- The elements are stored **sequentially in memory**, enabling **constant-time (O(1)) random access** via an integer index. The memory address of an element is calculated as: base_address + (index * element_size).
- **Indexing starts at 0** and goes up to **(length - 1)** in java.
- The size is **immutable** after creation.

Limitation: size cannot change. Inserting/deleting in the middle is costly (shifting required).

Array

One-dimensional array

A simple linear sequence of values. Access is direct: the index directly maps to a memory offset.

Multidimensional arrays in Java

- Java implements multidimensional arrays as **arrays of references** to other arrays (arrays of arrays).
- For a 2D array, the outermost array holds **references** to separate row arrays. These row arrays are themselves objects stored on the heap, so the memory for the whole 2D structure is **not necessarily contiguous**.
- Accessing an element a[i][j] requires **two levels of indirection**: first, the reference at index i of the outer array is retrieved, then index j of that inner array is used.
- This design allows **jagged arrays** – rows can have different lengths – because each inner array is independently allocated.

ArrayList

- Resizable-array implementation of the List interface.
- **Fast random access** (O(1)) by index.
- Adding elements at the end is amortised O(1), while inserting/removing inside the list requires shifting (O(n)).
- Only stores objects; uses generics for type safety.
- Provides many utility methods: add, get, remove, contains, sort.
- Grows dynamically – no need to predefine capacity.

LinkedList

- Doubly-linked list implementation of both List and Deque.
- **Efficient insertion/deletion** at the ends or with an iterator – O(1).
- Access by index is O(n) because the list must be traversed.
- Can be used as a lightweight queue or stack.
- Key methods: addFirst, addLast, getFirst, getLast, removeFirst, removeLast.

Stack

- Last-In-First-Out.
- The legacy Stack class exists, but Deque implementations (like ArrayDeque) are preferred.
- Core operations: **push** (add to top), **pop** (remove from top), **peek** (view top without removing).

Queue & Deque

Queue (FIFO)

- First-In-First-Out.
- Main operations: **offer** (add to end), **poll** (remove from front), **peek** (view front).
- Common implementations: LinkedList, ArrayDeque.

Deque (double-ended queue)

- Allows insertion and deletion at both ends.
- Operates as addFirst / addLast, removeFirst / removeLast.
- Can be used as either a queue or a stack.
- Preferred general-purpose implementation: ArrayDeque.

Set – HashSet

- Collection that **disallows duplicates**.
- No order is guaranteed.
- Average O(1) time for basic operations: add, remove, contains.
- Relies on hashCode() and equals() for object comparison.

Map – HashMap

- Stores **key-value pairs**, keys must be unique.
- No ordering guarantee.
- Average O(1) for put, get, remove, containsKey.
- Relies on hashCode() and equals() of the key objects.
- One null key allowed, multiple null values allowed.

Choosing the Right Data Structure

Requirement	Suggested Structure
Frequent random access	ArrayList, Array
Many insertions/deletions inside	LinkedList
Fast lookup, no duplicates	HashSet
Sorted elements	TreeSet, TreeMap *
Key-value pairs	HashMap
LIFO	Deque (as stack)
FIFO	Queue (ArrayDeque, LinkedList)